

Hibernate ddl-auto Values

The **ddl-auto** property tells Hibernate what to do with the database schema when the application starts. The schema includes tables, columns, primary keys, foreign keys, and other database objects. Choosing the correct value is important because some values only check the database, while others can change or delete it.

```
spring.jpa.hibernate.ddl-auto=value
```

1. validate

What it does: Hibernate checks whether the existing database schema matches the entity mappings. It does not create, update, or delete anything.

How it works: During startup, Hibernate reads the entity classes and compares the expected tables, columns, and data types with the database. If an important difference is found, the application fails to start with a schema validation error. If everything matches, startup continues normally.

Example: A Customer entity has an email field, but the customers table does not have an email column. Hibernate detects the mismatch and stops the application.

```
spring.jpa.hibernate.ddl-auto=validate
```

Real-world case: This value is useful when database changes are managed by migration scripts or a database team. It can be used in staging or production to make sure that the application version is compatible with the deployed database before accepting requests.

2. update

What it does: Hibernate tries to adjust the existing schema so that it matches the entity mappings. Existing data is usually kept.

How it works: At startup, Hibernate compares the mappings with the database and generates DDL statements when something is missing. It may create a new table or add a new column. However, it does not safely handle every change, such as renaming columns, removing old columns, or transforming existing data.

Example: A phoneNumber field is added to the Customer entity. Hibernate may execute an ALTER TABLE statement to add the related column.

```
spring.jpa.hibernate.ddl-auto=update
```

Real-world case: It is convenient for local development, learning projects, and prototypes because the database follows simple entity changes automatically. It is normally avoided in important production systems because the generated changes are not reviewed and may behave differently on different databases.

3. create

What it does: Hibernate rebuilds the schema every time the application starts. Existing tables managed by Hibernate are dropped and created again, so their data is lost.

How it works: Hibernate reads the entity mappings, removes the old schema objects, and generates new CREATE TABLE statements. The new schema reflects the current entity classes.

Example: If the customers table contains 100 records, restarting the application with create can remove the table, create it again, and leave it empty.

```
spring.jpa.hibernate.ddl-auto=create
```

Real-world case: It is suitable for a disposable local database, a classroom example, or a test that needs a clean schema at startup. It must not be used with valuable or permanent data.

4. create-drop

What it does: Hibernate creates a fresh schema when the application starts and drops it when the application shuts down normally.

How it works: The startup behavior is similar to create. Hibernate builds the required tables from the entity mappings. When the EntityManagerFactory or SessionFactory closes, Hibernate sends DROP statements to remove the generated schema.

Example: An integration test starts with empty Customer and Order tables, inserts temporary records, and removes the tables when the test application closes.

```
spring.jpa.hibernate.ddl-auto=create-drop
```

Real-world case: It is useful for integration tests, temporary demo environments, and in-memory databases. It keeps temporary test data isolated. It is not suitable for any database that must keep its data after shutdown.

5. none

What it does: Hibernate performs no automatic schema management. It does not validate, create, update, or drop database objects.

How it works: The application uses the database exactly as it already exists. The schema must be prepared manually, by reviewed SQL scripts, by a migration tool, or by a database team. If the schema is incorrect, the error may appear later when a query is executed.

Example: Adding a new field to an entity does not add a database column. The column must be created separately before the application uses that field.

```
spring.jpa.hibernate.ddl-auto=none
```

Real-world case: This is a common production choice when Flyway, Liquibase, SQL scripts, or a DBA controls every schema change. It prevents Hibernate from changing the database unexpectedly.

Quick Comparison

Value	What it does	Real-world case
validate	Checks that the existing schema matches the entities. It makes no changes.	Staging or production with controlled database migrations.
update	Adds or adjusts schema objects while usually keeping existing data.	Local development, learning projects, and prototypes.
create	Drops and recreates the schema at every startup.	Disposable databases, demos, or tests that need a clean start.
create-drop	Creates the schema at startup and drops it at normal shutdown.	Integration tests and temporary or in-memory databases.
none	Performs no automatic schema action.	Production managed by Flyway, Liquibase, SQL scripts, or a DBA.

Simple recommendation: Use update for small local development projects, create or create-drop for disposable test databases, and validate or none for controlled production environments.